Tastelet: Project Scope & Specification

Oct 10, 2026 · @Billy

Overview and goals

Tastelet is a Netflix-style movie recommender web app: the user picks favorite movies, and a neural network returns a taste persona, a favorite-genre sentence and a list of recommended movies. It is built solo for the AI course, and its main syllabus topic is **neural networks** (week 11-12), supported by clustering (week 14).

Goals

- Deliver a working web app with one core flow: pick favorites, then see persona, genre sentence and recommendations.
- Show with numbers that the neural model beats simple baselines (popularity, item-based KNN).
- Keep every AI step explainable in the course report.

Success looks like

- The neural model beats the popularity baseline on Hit@10 and RMSE on the test set; if it only ties matrix factorization or KNN, that is reported honestly.
- A new user gets results in under 3 seconds after submitting picks.
- The demo runs end to end from a deployed URL with no manual steps.

Scope

Version 1 is one flow with no accounts; everything else is out. Scope is frozen: new ideas go to a "later" list, not into the build.

In scope (v1)	Out of scope
Home grid of popular, trending and all-time movies	Search bar
Pick 5 or more favorite movies (counts as 5.0 ratings)	"Not for me" / dislike button

In scope (v1)	Out of scope
Neural collaborative filtering model trained on MovieLens	User accounts, login, saved history
Cold-start fold-in for a new anonymous user	Database for user data
K-means personas from trained user embeddings	Admin panel or manual review tools
Favorite-genre sentence from a template	LLM-generated text
Recommendations with match scores	Brand-new releases the model has not seen
Baselines and evaluation for the report	Music, anime or other domains
Deployed backend and frontend	Mobile app, multi-language UI (English only)

User flow and features

The user moves through three screens: browse, pick, result. Each feature has acceptance criteria that define when it is done.

1. **Home:** the app shows a poster grid of popular, trending and all-time movies.
2. **Pick:** the user taps posters to select favorites, with a live counter.
3. **Result:** the app shows a persona, a genre sentence and recommended movies.

ID	Feature	Acceptance criteria
F1	Home grid	Shows three sections (trending, popular, all-time) with posters and titles; all movies shown exist in the model's data; loads in under 2 s
F2	Movie picker	Tap selects or deselects a poster; counter reads "n/5"; minimum 5 and maximum 10 picks; submit is disabled below 5
F3	Recommendations	Returns 10 to 20 unseen movies, ranked by predicted score, each with poster, title and match score; picked movies never appear

ID	Feature	Acceptance criteria
F4	Persona	Shows one persona name and a one-line description from a fixed set of about 8; same picks always give the same persona
F5	Genre sentence	Names the top 1-2 genres from the picks and the most common decade, e.g. "You gravitate toward Crime and Thriller, especially films from the 1990s"
F6	Restart	A "Start over" button clears picks and returns to the home grid

Screen content

- **Home:** title, short instruction ("Pick movies you love"), three poster sections, sticky counter and "See my results" button.
- **Result:** persona name as the large heading, description and genre sentence below it, a poster row of recommendations with match scores, and the Start over button.

AI/ML specification

The AI work has five parts: data split, baselines, the neural model, cold-start fold-in and persona clustering. Only these are documented in the course report.

Component	Specification	Syllabus topic
Data	MovieLens `ml-latest-small` (about 100k ratings, 610 users, 9.7k movies); user and movie IDs re-indexed to 0..N-1; random 80/10/10 train/validation/test split	-
Baselines	Global mean rating; popularity (most-rated movies); item-based KNN with cosine similarity	Classification (KNN)
Neural model	Neural collaborative filtering in PyTorch: user and movie embeddings (dim 32), concatenated, then MLP 64 -> 32 -> 1 with ReLU and dropout 0.2; MSE loss, Adam, about 20 epochs, keep the best validation epoch	Neural networks

Component	Specification	Syllabus topic
Cold-start fold-in	Create a new user vector, freeze all trained weights, treat each pick as a 5.0 rating, optimize only that vector for about 100 steps, then score all unseen movies	Neural networks
Personas	K-means on trained user embeddings, K chosen from 6-12 by silhouette score and interpretability (expected about 8); each cluster named by hand once; new user assigned to the nearest centroid	Clustering
Genre sentence	Count genres in the picks, take top two plus most common decade, fill a fixed template	None (statistics)

Evaluation metrics

- **RMSE** on the test set for rating prediction.
- **Hit@10** (or Recall@10): hide one liked movie per test user and check whether it appears in the model's top 10.
- All three baselines and the neural model are evaluated on the same split, and results go into one comparison table in the report.

Tuning plan

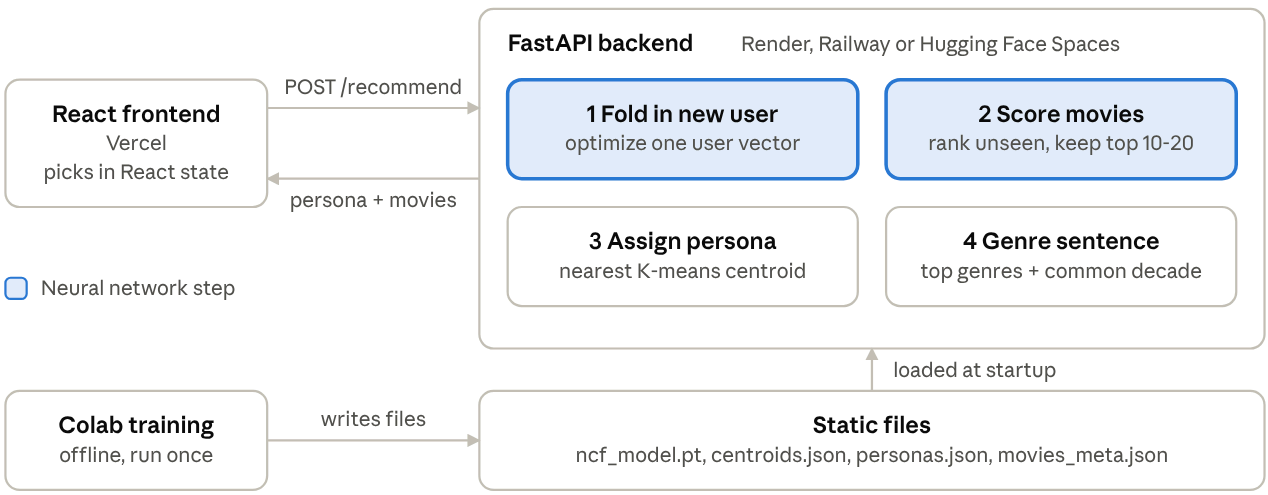
- Try embedding size 16, 32 and 64, dropout 0.1 to 0.3 and learning rate 1e-3 and 1e-2; record each run in a results table.
- Fix the random seed so results are reproducible.

Model outputs saved to disk

- `ncf_model.pt` (trained weights), `centroids.json` (cluster centers), `personas.json` (names and descriptions), `movies_meta.json` (titles, genres, years, posters).

Architecture, tech stack and API

The app is a stateless two-part system: a React frontend and a Python FastAPI backend that holds the trained model in memory.



system architecture · 2 services, 1 offline job

Each request runs four steps in order; steps 1 and 2 are the neural network, and the files are produced offline in Colab.

Layer	Choice	Hosting
Frontend	React (Vite), poster grid, result page; picks held in React state	Vercel
Backend	Python, FastAPI, PyTorch on CPU, scikit-learn, pandas	Render, Railway or Hugging Face Spaces (Vercel does not run PyTorch well)
Training	Google Colab notebook; outputs saved as model and JSON files	-
Storage	Static files bundled with the backend; no database	-

API endpoints

Endpoint	Request	Response
`GET /movies`	none	Home grid data: `trending`, `popular` and `all_time` lists, each movie with `id`, `title`, `year`, `genres`, `poster`

Endpoint	Request	Response
`POST /recommend`	`{"movie_ids": [int, ...]}` with 5 to 10 IDs	`{persona, description, genre_sentence, recommendations: [{id, title, poster, score}]}`
`GET /health`	none	`{"status": "ok"}`, used to wake a sleeping free backend

Errors: fewer than 5 or more than 10 picks, or an unknown movie ID, returns a 422 with a plain message that the frontend shows to the user.

Data handling

All data is prepared once, offline, and shipped with the backend as static files; the running app never calls an external API and stores nothing about users.

Source	Used for	Notes
MovieLens `ml-latest-small` (grouplens.org)	Ratings, titles, genres, links	Includes `links.csv`, which maps to TMDB IDs; the 1M version has no links file. Can switch to `ml-25m` later with the same code
TMDB API (free key)	Posters, overviews, trending list	Fetched once into `movies_meta.json`; trending list refreshed manually before the demo

Home grid lists

- **All-time:** movies with at least 50 ratings, ranked by average rating.
- **Popular:** movies ranked by number of ratings.
- **Trending:** TMDB trending, filtered to movies that exist in the MovieLens data.

Rules

- Only movies present in both datasets are shown, because the model cannot score unseen titles.
- Attribute MovieLens and TMDB in the app footer, as their terms require.
- Keep the TMDB key in an environment variable, never in the frontend or the repository.

Non-functional requirements

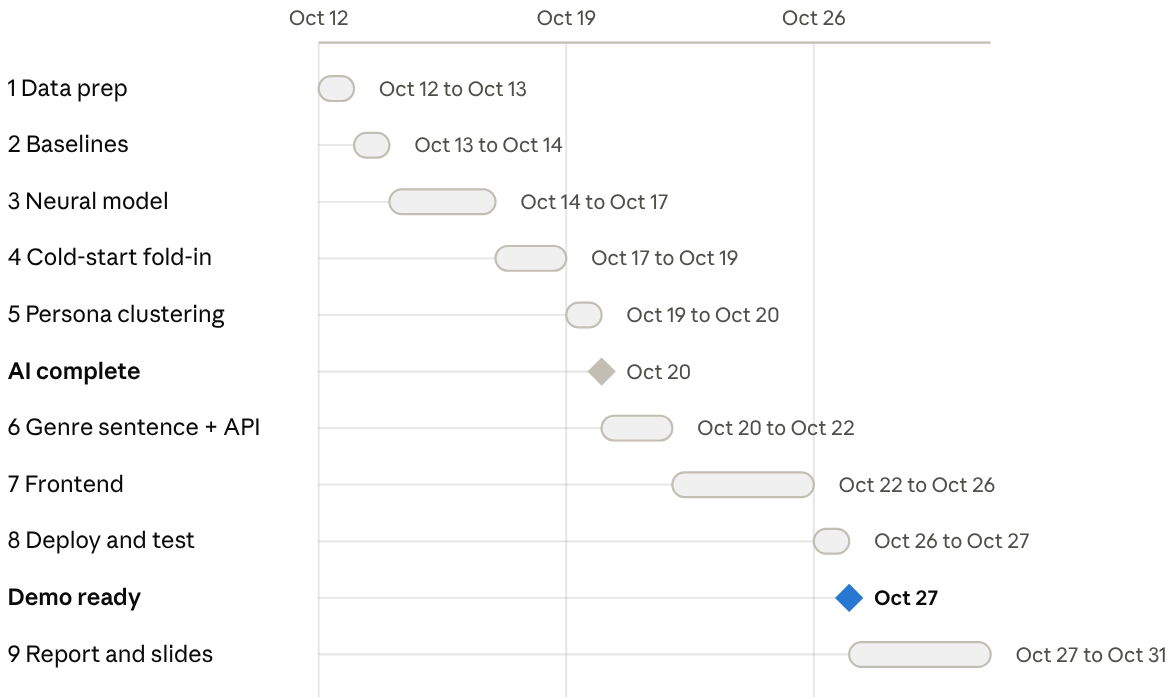
Area	Requirement
Performance	Recommendation response under 3 s on CPU; model, centroids and metadata loaded once at startup
Privacy	No accounts, no database, no cookies beyond the browser session; picks are sent only to compute results and are not stored or logged
Reliability	If the backend is asleep (free hosting), the frontend shows a loading message instead of an error; invalid requests return clear 4xx messages
Usability	Works on phone and desktop; poster grid readable at 360 px width; English UI only
Reproducibility	Fixed random seed; `requirements.txt` pinned; one notebook or script reruns the whole training pipeline
Cost	Everything runs on free tiers (Colab, Render or Hugging Face Spaces, Vercel)
Maintainability	AI code and app code in separate folders; README explains setup in under 10 steps

Timeline and milestones

The plan runs about 20 calendar days from an assumed start on Monday, October 12, with all AI work done by October 20 and the demo ready on October 27. Shift the dates once the course deadlines are known.

AI work is done by Oct 20 and the demo is ready by Oct 27

Solo plan from Mon Oct 12, calendar days; shift once course deadlines are known



project timeline · 9 phases, 2 milestones

Diamonds mark the two milestones: AI complete (Oct 20) and Demo ready (Oct 27), the date the plan protects.

Exit criteria per phase (tick when met)

- [ ] **1 Data prep:** ratings, movies and links loaded; IDs re-indexed; split saved; `movies_meta.json` built with posters
- [ ] **2 Baselines:** global mean, popularity and item-KNN evaluated on the test set (RMSE, Hit@10)
- [ ] **3 Neural model:** NCF trained, best epoch saved, loss curves plotted, beats popularity on both metrics
- [ ] **4 Cold-start fold-in:** 10 sample users tested; recommendations look sensible and exclude the picks
- [ ] **5 Persona clustering:** K chosen, every cluster named by hand, `centroids.json` and `personas.json` saved
- [ ] **6 Genre sentence + API:** `/movies`, `/recommend` and `/health` work locally and return the agreed format

- [ ] **7 Frontend:** home grid, picker with counter, result page and Start over button work on phone and desktop
- [ ] **8 Deploy and test:** both services live; full flow tested from a fresh browser; response under 3 s
- [ ] **9 Report and slides:** every item in the report checklist below is done

Risks and mitigations

Risk	Likelihood	Mitigation
Neural model barely beats baselines	Medium	Report honestly with a clean evaluation; try matrix factorization as an extra comparison; tune embedding size
Cold-start fold-in gives odd results for niche picks	Medium	Mix popular and less obvious movies in the grid; test 10 sample users early and inspect results
Personas feel generic or overlap	Medium	Try several K values; name clusters only after reading their top genres and movies; merge clusters that look alike
New releases missing from the data	High	Filter the home grid to known movies (already in scope); mention as a limitation
Free backend sleeps or is slow	Medium	Show a loading state; ping the backend before the demo; keep the model small
TMDB poster or ID mismatches	Low	Fall back to a placeholder poster; drop movies without a valid link
Scope creep (search, accounts, dislike button)	Medium	Scope is frozen; extra ideas go to a "later" list
Time runs short	Medium	Phases 1-5 contain all the AI and come first; frontend polish and optional features come last

Course deliverables and definition of done

The project is done when the checklist below is fully ticked.

Report contents (AI only, not the web stack)

- [ ] Problem statement and why neural networks fit recommendation
- [ ] Dataset description and split
- [ ] Comparison table: global mean vs. popularity vs. item-KNN vs. neural net (RMSE and Hit@10)
- [ ] Training and validation loss curves
- [ ] Cold-start fold-in explanation with 2-3 worked examples (picks in, persona and recommendations out)
- [ ] Cluster table: persona names with top genres and example movies
- [ ] Limitations (new releases, small dataset, no dislike signal) and future work

Demo and delivery

- [ ] Deployed frontend and backend, tested end to end from a fresh browser
- [ ] Demo script of about 3 minutes: pick 5 movies, show persona and recommendations, repeat with a very different taste
- [ ] Presentation slides (course schedule: week 15)
- [ ] Source code in one repository with a README and the training notebook

Definition of done

- [ ] Features F1-F6 meet their acceptance criteria
- [ ] Neural model evaluated against all three baselines on the same split
- [ ] Response time under 3 s on the deployed backend
- [ ] No out-of-scope features added

Open question: the exact report and presentation deadlines are not set here yet; add the dates to the timeline once the course announces them.